02 / 38 — Phase 2 Assessment: Rendering, Reconciliation, and Internals

0. How to use this assessment

Covers: lessons 01–37 of Phase 2 (render triggers, render/reconcile/commit/paint, mount vs update, virtual DOM, elements vs DOM nodes, reconciliation and keys, Fiber, work-in-progress trees, render vs commit, purity, scheduler and lanes, interruptible rendering, concurrent misconceptions, hooks on fibers, effects and their timing, commit and DOM mutation, paint, batching, transitions, Suspense, hydration, SSR, RSC model, Strict Mode, unnecessary renders, Profiler, reading React's source, and Scenarios A–F).

Format: closed-notes first pass, then open-docs only for the implementation tasks. React 19.x, TypeScript 5.x, StrictMode **off** unless a question says otherwise. Allow 5–6 hours in total; do not split the prediction section across days (you need the mental model fresh).

Rules for honest self-grading

1. Write every answer down before reading Section 8 (the answer key). "I knew that" does not count.
2. For prediction tasks you must write the full log or DOM state; partial answers get partial credit per the key.
3. For implementation tasks, **run** your code and the tests. Code that was never executed is graded at most half.
4. Questions touching version-sensitive behavior are marked ⚠. State the version you assume; the key distinguishes documented guarantees from implementation details. (The key's predictions R1-R6, D1, D2, D4 and the I1-I4 solutions were executed against react/react-dom 19.3.1 on 2026-10-22; where an item is still version-sensitive the key says so.)

Categories and points (each category totals 100; assessment total 400)

Category	Where	Points
Explanation	Part 1: E1–E10 (10 × 10)	100
Reasoning	Part 2: R1–R6 predictions (6 × 10) + Part 5: R7–R9 architecture (14 + 13 + 13)	100
Debugging	Part 3: D1–D5 (5 × 20)	100
Implementation	Part 4: I1–I4 (4 × 25)	100

Part 1 — Explanation (100 points)

Answer each in your own words in 120–250 words. Each question is scored by key points (see key).

E1. (10) Distinguish *invoking* a component function, *rendering*, *reconciling*, *committing*, running *layout effects*, *painting*, and running *passive effects*. State which are interruptible, which touch the DOM, and

which are guaranteed to happen before the browser paints. Give a concrete situation in which a component renders but nothing is committed.

E2. (10) "React uses a virtual DOM, which is why it is fast." Evaluate this sentence. Explain what a React element is, what a fiber is, what a DOM node is, and what React does *not* do when it "diffs".

E3. (10) State the reconciliation heuristics React uses, explain why they exist, and give three realistic code patterns that cause **unintended state reset** and three that **intentionally** reset state.

E4. (10) Why was the stack reconciler replaced? Explain how a fiber works as a unit of work, how the current and work-in-progress trees implement double buffering, and what capabilities this architecture enabled.

E5. (10) Why must render be pure? Connect your answer to interruptible rendering, Suspense, StrictMode's double invocation, and memoization. What is *allowed* inside render that looks impure?

E6. (10) Explain urgent vs transition updates, lanes (conceptually), time slicing, and what cannot be interrupted. Explain how you would verify a claim about the scheduler in React's source and what caveats apply.

E7. (10) Explain layout effects vs passive effects: timing relative to DOM mutation and paint, execution order across a parent/child tree (mount and update), cleanup order, and what changes in StrictMode development.

E8. (10) Explain automatic batching, the "snapshot" semantics of state, updater functions, how hooks are stored on a fiber and why the Rules of Hooks follow from that, and when `flushSync` is legitimate.

E9. (10) Explain what "suspending" does to the render phase, how it differs on initial mount vs on update in a revealed boundary vs inside a transition, and what must hold about the promise passed to `use`.

E10. (10) Explain SSR, hydration, hydration mismatch handling (what is an error vs a warning), selective hydration, and how React Server Components differ from SSR. State what is *not* hydrated.

Part 2 — Prediction tasks (60 points of the Reasoning category)

Assume React 19, `createRoot`, **no StrictMode**, browser event handlers triggered by real user clicks/typing unless stated.

R1. Effects order (10)

```tsx
import { useEffect, useLayoutEffect, useState } from 'react';

function Child({ n }: { n: number }) {
  console.log('Child render', n);
  useLayoutEffect(() => {
    console.log('Child layout', n);
    return () => console.log('Child layout cleanup', n);
  });
  useEffect(() => {
    console.log('Child effect', n);
    return () => console.log('Child effect cleanup', n);
  });
  return <span>{n}</span>;
}

function Parent() {
  const [n, setN] = useState(0);
  console.log('Parent render', n);
  useLayoutEffect(() => { console.log('Parent layout', n); });
  useEffect(() => {
    console.log('Parent effect', n);
    if (n === 0) setN(1);
  });
  return <Child n={n} />;
}

// createRoot(container).render(<Parent />);
```

Write the complete console output from mount until the system is idle.

R2. Batching and snapshots (10)

```
function Counter() {
  const [c, setC] = useState(0);
  console.log('render', c);
  return (
    <button
      onClick={() => {
        setC(c + 1);
        setC(c + 1);
        setC((x) => x + 1);
        console.log('handler sees', c);
        setTimeout(() => {
          setC((x) => x + 10);
          console.log('timeout sees', c);
        }, 0);
      }}
    >
      {c}
    </button>
  );
}
```

The user clicks **once**. Write the complete log (starting with the mount log) and the final button text.

R3. Keys, state, and DOM identity (10)

```
function Row({ label }: { label: string }) {
  const [note, setNote] = useState('');
  console.log('Row render', label);
  return (
    <li>
      <span>{label}</span>
      <input value={note} onChange={(e) => setNote(e.target.value)} aria-label={`note
${label}`} />
    </li>
  );
}

function List({ items }: { items: string[] }) {
  return <ul>{items.map((l, i) => <Row key={/* see below */ l} label={l} />)}</ul>;
}
```

`items` starts as `['A','B','C']`. The user types `x` into the **first** input, then the parent re-renders `List` with `['C','B','A']`. (`Row` is not memoized.)

For each of the three key strategies **(a)** `key={i}`, **(b)** `key={l}`, **(c)** `key={Math.random()}`, state: (1) the log lines produced by the re-render; (2) what the first input contains and next to which label; (3) how many

`li` DOM nodes are moved/re-inserted, and how many DOM nodes are created and removed; (4) what happens to focus if the first input had focus.

R4. `useDeferredValue` log (10)

```
import { memo, useDeferredValue, useState } from 'react';

const List = memo(function List({ q }: { q: string }) {
  console.log('List q=' + q);
  return <ul><li>{q}</li></ul>;
});

export function Search() {
  const [q, setQ] = useState('');
  const deferred = useDeferredValue(q);
  console.log(`Search q=${q} d=${deferred}`);
  return (
    <>
      <input value={q} onChange={(e) => setQ(e.target.value)} />
      <List q={deferred} />
    </>
  );
}
```

(a) Write the mount log. (b) The user types `a`, nothing else, and React is allowed to finish all work uninterrupted: write the new log lines and the number of commits. (c) Suppose the user types `b` while the deferred render for `a` is *in progress* (the `List` is expensive). Which lines appear in the log, in what order, and which are never committed? (d) State which part of your answer is a public guarantee and which is a teaching model.

R5. Suspense timelines (10)

```tsx
const cache = new Map<string, Promise<string>>();
function getData(id: string): Promise<string> {
  let p = cache.get(id);
  if (!p) { p = new Promise((r) => setTimeout(() => r(`data ${id}`), 1000)); cache.set(id, p);
}
  return p;
}

function Panel({ id }: { id: string }) {
  const data = use(getData(id));
  return <section><p>{data}</p><input aria-label="note" defaultValue="" /></section>;
}

function App() {
  const [id, setId] = useState('1');
  const [isPending, startTransition] = useTransition();
  return (
    <>
      <button onClick={() => setId('2')}>urgent</button>
      <button onClick={() => startTransition(() => setId('2'))}>transition</button>
      <Suspense fallback={<p>loading</p>}><Panel id={id} /></Suspense>
      {isPending && <p role="status">updating</p>}
    </>
  );
}
```

Initial state: `id='1'` fully loaded and visible (assume its promise already resolved). The user types `hi` into the note input, then clicks **(a) urgent** or **(b) transition** (separate runs). For each run describe, for the 1000 ms wait: what the user sees, how many commits occur and what each contains, what the DOM contains for the `section` and its input, and whether `hi` survives after loading. Then answer: (c) what changes for (a) and (b) if the first render of the whole app had suspended (i.e. initial mount with `id='1'` unresolved)?

R6. Hydration outcomes (10; 2.5 each)

For each pair, the server HTML is produced by `renderToString`, then the client calls `hydrateRoot`. Classify the outcome as: **M** (match, no error), **E** (recoverable error, client-render of boundary/root), **W** (warning only, server DOM kept), or **X** (match plus an extra client render). Give the final DOM text/attribute and justify.

1. `<p>{Math.random()}</p>`
2. `<div id={String(Math.random())}>x</div>`
3. `const id = useId(); <><label htmlFor={id}>L</label><input id={id} /></>`
4. `const [m, setM] = useState(false); useEffect(() => setM(true), []); <p>{m ? 'client' : 'server'}</p>`

Part 3 — Debugging challenges (100 points)

For each: (i) identify **every** defect, (ii) explain the mechanism using render/commit vocabulary, (iii) provide corrected code, (iv) state how you would verify the fix with DevTools or logs.

D1. Laggy input that sometimes reverts (20)

```
function SearchPage({ items }: { items: string[] }) {
  const [text, setText] = useState('');
  const [isPending, startTransition] = useTransition();

  const visible = items.filter((s) => s.includes(text));   // 20k items, ~5 ms
  return (
    <>
      <input
        value={text}
        onChange={(e) => startTransition(() => setText(e.target.value))}
      />
      {isPending && <span>…</span>}
      <HeavyList items={visible} />                          {/* ~400 ms render */}
    </>
  );
}
function HeavyList({ items }: { items: string[] }) {
  return <ul>{items.map((s) => <HeavyRow key={s} text={s} />)}</ul>;
}
```

Symptoms: characters appear late, occasionally vanish when typing fast.

D2. Forever loading (20)

```
function UserCard({ id }: { id: string }) {
  const user = use(fetch(`/api/users/${id}`).then((r) => r.json() as Promise<{ name: string
}>));
  return <h2>{user.name}</h2>;
}

export function Page({ id }: { id: string }) {
  return (
    <Suspense fallback={<p>Loading…</p>}>
      <UserCard id={id} />
    </Suspense>
  );
}
```

Symptoms: spinner never resolves; the Network tab shows hundreds of requests. A teammate "fixed" it with `useMemo(() => fetch(...), [id])` and it still loops on the first load. Explain why, fix it properly, and describe a second bug that will appear when the user navigates between ids (hint: which UI do they see and how would you change that?).

D3. Page flashes and an attribute is wrong (20)

```
function Page({ user }: { user: { name: string } | null }) {
  const theme = typeof window === 'undefined' ? 'light' : localStorage.getItem('theme') ??
'light';
  const now = new Date().toLocaleString();
  return (
    <div className={`page ${theme}`}>
      <p><div>Welcome {user?.name ?? 'guest'}</div></p>
      <small>Rendered at {now}</small>
    </div>
  );
}
```

Symptoms in production: first paint is correct HTML, then content flashes; some users keep the light theme although they selected dark; the console (dev) prints several messages.

D4. Impure render (20)

```tsx
let renderCount = 0;

function Leaderboard({ scores }: { scores: number[] }) {
  renderCount++;
  const sorted = scores.sort((a, b) => b - a);
  return (
    <ol>{sorted.map((s, i) => <li key={i}>{s}</li>)}<li>renders: {renderCount}</li></ol>
  );
}

function Summary({ scores }: { scores: number[] }) {
  return <p>First entered: {scores[0]}</p>;
}

export function Game() {
  const [scores, setScores] = useState<number[]>([10, 30, 20]);
  return (
    <>
      <Leaderboard scores={scores} />
      <Summary scores={scores} />
      <button onClick={() => setScores([...scores, 25])}>Add 25</button>
    </>
  );
}
```

Symptoms: the product owner expects "First entered: 10" (the first score entered), but the page shows `30`; the `renders:` counter increases by 2 per update in development; and if a developer swaps the order of `<Leaderboard>` and `<Summary>` the summary suddenly shows `10`. List every defect, explain each symptom, and fix.

D5. `memo` that does nothing (20)

```tsx
const Chart = memo(function Chart({ options, onSelect }: {
  options: { color: string; smooth: boolean };
  onSelect: (id: string) => void;
}) { /* expensive */ return <canvas />; });

function Dashboard({ rows }: { rows: Row[] }) {
  const [hover, setHover] = useState<string | null>(null);
  return (
    <div onMouseMove={(e) => setHover((e.target as HTMLElement).id)}>
      <Chart options={{ color: 'steelblue', smooth: true }} onSelect={(id) => selectRow(rows,
id)} />
      <Tooltip id={hover} />
    </div>
  );
}
```

The Profiler flame graph shows `Chart` rendering on every mouse move ("props changed: options, onSelect"). Explain, give three fixes with trade-offs (including a structural fix that needs no memoization and a mention of the React Compiler), and state under which circumstances you would *not* fix this at all.

Part 4 — Implementation exercises (100 points)

Write TypeScript that compiles under `strict`. Provide tests where requested. Run them.

I1. Render/commit instrumentation (25)

Implement `createRecorder()` returning `{ onRender, commits, reset }` for use with `<Profiler>`, typed with `ProfilerOnRenderCallback`. Then write a Vitest + React Testing Library test proving that a `memo` child is **not** re-rendered when its parent's unrelated state changes, using (a) a spy invoked from the child's body and (b) a Profiler-based assertion. State precisely what each assertion proves and one thing it cannot prove.

I2. Transition-based filter with accessibility (25)

Implement `FilterableList({ items }: { items: readonly string[] })`: the input must echo synchronously; the filtering applies via a transition; the list is memoized; while pending the list is visually dimmed and the container has `aria-busy`; a polite live region announces "Updating results" while pending. Include a test that types a query and eventually sees only matching items. Explain how you would prove in the browser that the list is not rendered in the urgent commit.

I3. Suspense-compatible keyed cache (25)

Implement `createKeyedCache<K extends string | number, V>(loader: (key: K) => Promise<V>)` with `get(key)` (stable promise identity per key), `preload(key)`, and `invalidate(key?)`. Then write `Section`: an error boundary (class component) + `Suspense` + a consumer using `use(cache.get(id))`,

with a "Retry" button that invalidates and resets the boundary. Explain what happens if a rejected promise is evicted automatically.

I4. Hydration-safe media query hook (25)

Implement `useMediaQuery(query: string, serverValue?: boolean): boolean` using `useSyncExternalStore` such that SSR and hydration use `serverValue` and the client value is applied after hydration without a mismatch error. Write a jsdom test using `renderToString` + `hydrateRoot` that asserts: no `onRecoverableError` calls, initial server HTML reflects `serverValue`, final DOM reflects the mocked `matchMedia`. Explain why `subscribe` must be stable.

Part 5 — Architecture and reasoning (40 points of the Reasoning category)

R7. Dashboard rendering architecture (14)

A dashboard: a global filter bar (date range, region, search), 12 widgets (8 canvas charts, 2 server tables of ~5,000 rows, 2 KPI tiles), a WebSocket price ticker updating 10×/second, route-based navigation between three dashboards. Requirements: the filter bar must never feel laggy; widgets may update with a delay but must not shift layout; one failing widget must not blank the page; navigation keeps the old page until the new one is ready; initial HTML must show the layout skeleton. Design the state placement, use of transitions/deferred values, Suspense and error boundaries, memoization strategy, ticker integration, SSR/hydration approach, and the measurement plan. Identify two decisions where you would explicitly *not* use a concurrent feature and why.

R8. SSR product page (13)

A product detail page for an international shop: price depends on country (cookie), a "N people are viewing" badge, a cart count stored in `localStorage`, an A/B test variant stored in a cookie, reviews fetched slowly (1.5 s), and a countdown to the end of a sale. For each element decide: server-rendered HTML, after-mount update, or streamed behind Suspense; list the hydration mismatch risks and the specific mitigation for each; describe selective hydration's role; describe error reporting and how you would detect HTML/JS skew after a deploy.

R9. "Too many re-renders" governance (13)

A VP asks you to "make React stop re-rendering so much", and a team proposes wrapping every component in `memo` / `useMemo` / `useCallback`. Write the policy you would propose: an investigation procedure (tools and evidence), decision criteria for when to memoize, restructure state, or accept re-renders, how the React Compiler (react-compiler 1.0.0 is on npm as of 2026-10-22; check the current docs) changes the policy, and how you would prevent regressions. Include at least one case in which memoization makes things worse.

Answer key

> Attempt the assessment first. Everything below is answers, grading guidance, and the rubric.

---

6. Part 1 — Key points and model answers

Score each question: 2 points per key point achieved (partial = 1). Up to 5 key points.

E1

Key points:

1. **Invocation/rendering = calling the component function** (and children) in the render phase; it produces elements; it does not touch the DOM. Reconciliation (comparing new elements to the current fibers and marking work) happens in the same phase.
2. The **render phase is pure, interruptible (in concurrent lanes), can be repeated or discarded**; commit is not.
3. **Commit** is synchronous: DOM mutations, refs, **layout effects** (after mutations, before the browser paints; synchronous so they block paint).
4. **Paint** is the browser's job after the JS task yields; passive effects normally run after paint (React may flush them earlier, e.g. before the next render, or synchronously at the end of the commit for an update caused by a discrete event such as a click: documented in the React 18 upgrade guide as "consistent useEffect timing" and observed on 19.3.1).
5. A concrete render-without-commit: a transition render abandoned by a newer urgent update; or a render that suspends in a transition; or StrictMode's second invocation; or a state update that bails out after rendering.

Model answer: *Rendering is React calling your components to learn what UI should exist; nothing is visible yet. Reconciling compares the result with the current tree and records what must change. Commit applies the changes (DOM writes, ref attach, layout effects) in one synchronous step; the browser then lays out and paints; passive effects follow. Because only commit has side effects on the outside world, React can drop renders freely.*

Wrong answers that cost points: "render means the DOM is updated"; "useEffect runs before paint always"; "reconcile and render are different threads".

E2

Key points:

1. A **React element** is an immutable plain object (type, props, key, ref) *describing* UI; created by JSX/ `createElement`.
2. A **fiber** is the internal mutable work node (instance state, hooks, child/sibling/return pointers, flags); the "virtual DOM" in practice is elements + fibers; there's no single "virtual DOM" API.
3. A **DOM node** is the real host object owned by the browser; React holds references (`stateNode`), and `ref`s expose them.
4. React does **not** diff against the real DOM (it compares the new element tree with the previous fiber tree) except in hydration where it adopts existing DOM; it does not guarantee being faster than

hand-written DOM updates.
5. The benefit is a declarative model with batched, minimal commits and the ability to schedule/discard work; "fast" claims need measurement.

E3

Key points:

1. Heuristics: elements of **different types** at the same position → tear down and rebuild the subtree; **same type** → update in place and recurse; **children lists** are matched by **key** (then by index when no key); a linear-time compromise instead of generic tree-edit distance.
2. Identity = type + position (+ key). State lives with the fiber, so identity determines state survival.
3. Unintended reset: component defined inside another component (new type each render); conditional wrapper changing the parent type (`cond ? <div><X/></div> : <X/>`); changing `key` due to unstable keys (`Math.random()`, `key={item.name}` that changes on edit); moving a component to a different parent position.
4. Intentional reset: `key={userId}` on a form/profile; switching `type`; conditional render of a different element to remount.
5. Index keys on reorderable lists mismatch state to items; correct fix is stable ids; explains why diffing is O(n).

E4

Key points:

1. Stack reconciler: recursive, synchronous, cannot pause; long updates monopolize the main thread.
2. A fiber = one unit of work; the tree is traversed with explicit child/sibling/return pointers in a loop rather than recursion, so React can stop after any fiber and resume.
3. `current` (committed) and `workInProgress` tree linked by `alternate`; render builds WIP; commit swaps the root pointer; discard is cheap.
4. Enabled: priorities/lanes, time slicing, Suspense, error boundaries, transitions, selective hydration, offscreen.
5. Costs: more complex internals; effects of purity requirements on user code.

E5

Key points:

1. React may call render multiple times, discard results, or pause; side effects in render would repeat or leak.
2. StrictMode double-invokes render (and function initializers/updaters/reducers) in dev to expose impurities; also re-runs effects (mount/unmount/mount).
3. Suspense and transitions abandon work: any mutation done during an abandoned render is a bug (e.g. mutating props/state/module variables).
4. Memoization/bailouts (and the Compiler) assume same inputs → same output.
5. Allowed: local mutation of values created in the same render (array built and pushed), reading props/state/context, lazy initialization of refs (`ref.current ??=` pattern in specific documented ways), logging in dev. Effects and event handlers are where side effects belong.

E6

Key points:

1. Updates get lanes; discrete events are urgent; `startTransition` / `useDeferredValue` produce lower-priority transition work.
2. Time slicing: React yields between fibers (≈ 5 ms in the scheduler source, an implementation detail) so browser can paint/handle input; one main thread; no parallelism.
3. Not interruptible: commit; a single component's function execution; synchronous-lane renders (flushSync, discrete urgent updates).
4. An interrupted transition render is discarded and restarted from scratch with the latest state; logs for abandoned renders still appear; Profiler shows only commits.
5. Source verification: read `react-reconciler` and `scheduler` packages in `github.com/facebook/react`, search by concept (lanes, `shouldYield`, `performConcurrentWorkOnRoot`-style names are *not guaranteed stable*: that name no longer exists on `main`/19.3.1, where `performWorkOnRoot` replaced it), pin a version tag, note that names/files change between releases and that DevTools and tests are better evidence than naming memory.

E7

Key points:

1. Layout effects: during commit after DOM mutations, before paint, synchronous; setState inside forces synchronous re-render before paint. Passive: after commit, typically after paint, asynchronous (flush timing nuances: sync flush after discrete events, and `flushSync` may run pending effects).
2. Order on mount: children before parents within each category; all layout effects before passive effects.
3. On update: layout cleanups run during mutation (child before parent), then layout effects; passive: **all** cleanups first, then **all** creates, children first.
4. Cleanup runs before the next effect and on unmount; effects never run on the server.
5. StrictMode (dev): extra mount→unmount→mount cycle for effects (also ref callbacks in React 19, verified on 19.3.1), double render.

E8

Key points:

1. React 18+ batches updates from any source (events, timeouts, promises) into one render per tick/lane; `flushSync` forces a synchronous flush.
2. State is a snapshot per render; `setX(x+1)` repeated uses the same `x`; updater `setX(p=>p+1)` queues functions applied in order.
3. Hooks are stored as a linked list on the fiber in call order; each hook call reads the next cell → order must be stable → no conditional/looped hooks.
4. Update queue applies in order; rebasing across lanes (teaching model; the count-1 example rendered 2 then 4 on 19.3.1); `Object.is` bail-out.
5. `flushSync` legitimate for integrating with non-React code that reads DOM immediately (scroll into view after insert), rarely; costs performance and defeats batching.

E9

Key points:

1. Suspending abandons the render attempt of that subtree; React unwinds to the nearest boundary, attaches a ping, renders fallback, and commits atomically.
2. Mount: no state/DOM existed; fallback inserted then replaced. Update in revealed boundary (non-transition): existing DOM hidden (`display:none`), fallback inserted, state preserved, layout effects torn down (passive effects are not torn down by a Suspense hide, checked on 19.3.1). Transition: old UI stays, no fallback for revealed boundary, `isPending` true; new boundaries may still show fallback.
3. Promise identity must be stable across renders (cache keyed by params, created outside render, or via Suspense-compatible library); otherwise infinite suspend.
4. Rejected promise throws to error boundary; Suspense does not handle errors.
5. React 19 commits fallback immediately and pre-warms siblings (upgrade guide; reproduced on 19.3.1); 18 behaved differently; don't depend on sibling order. Also: reveals are throttled (about 300 ms after the previous reveal), so a promise that resolves at 100 ms is not necessarily visible at 100 ms.

E10

Key points:

1. SSR renders to HTML on the server (components run once, no effects); browser paints; hydration is a client render that adopts the existing DOM and requires identical output.
2. Mismatches: text/structure → recoverable error and client-render of the boundary/root (reported via `onRecoverableError`); attributes/styles → dev warning, not patched.
3. Suspense boundaries are independent hydration units; selective hydration prioritizes the boundary a user interacts with (a click on a dehydrated boundary triggers synchronous hydration of that boundary, after which the event is dispatched; if the boundary cannot hydrate yet the discrete event is dropped; only continuous events such as hover/focus are queued and replayed: `ReactDOMEventListener.js` / `ReactDOMEventReplaying.js`, executed on 19.3.1); streaming sends HTML progressively.
4. RSC: components rendered on the server only; output is a serialized payload (RSC payload), not HTML-hydration; server components have no state/effects/handlers and ship no client JS; client components (`'use client'`) are still SSR'd/hydrated.
5. Server components are **not hydrated**; SSR and RSC are orthogonal; bundler/framework integration details must be checked against current docs (react.dev: the RSC bundler APIs do not follow semver between 19.x minors).

---

7. Part 2 — Prediction answers

R1 answer (10 points)

```
Parent render 0
Child render 0
Child layout 0
Parent layout 0
Child effect 0
Parent effect 0
Parent render 1
Child render 1
Child layout cleanup 0
Child layout 1
Parent layout 1
Child effect cleanup 0
Child effect 1
Parent effect 1
```

Scoring: (2) render order parent→child both times; (2) layout effects before passive, child before parent; (2) update: layout cleanup before new layout effects; (2) passive cleanups happen before *any* passive effect create of the update; (2) the second render is triggered by the `setN(1)` in the passive effect and happens after the mount's passive effects ran. Reasoning: commit runs layout effects synchronously children-first; passive effects run later and the `setN(1)` schedules a new render. In the update commit, `Parent` has no cleanups; `Child` cleanups run first (layout cleanup during the mutation phase, passive cleanup at the start of the passive flush), then the new effects run children-first. (Executed on 19.3.1: the log above, line for line.) The exact interleaving of `Child effect cleanup 0` relative to *layout* effects of the update is as given because passive cleanups only run in the passive flush after the layout phase. Wrong answer "Parent layout before Child layout": effects run depth-first, children complete before parents.

R2 answer (10 points)

```
render 0
handler sees 0
render 2
timeout sees 0
render 12
```

Final text: **12**. Reasoning: all three updates in the handler are batched into one render: `c+1` (1), `c+1` (1; same snapshot), updater applied to 1 → 2. The handler logs the snapshot `0`. The timeout closure still holds `c = 0` (closures capture the render's snapshot), logs `0`; `setC(x => x + 10)` schedules a render with 2 + 10 = 12 (updater reads the latest state). In React 18+ the timeout update is batched too; `render 12` happens after the timeout callback finishes. Scoring: 2 for `render 2` (not 1, not 3); 2 for snapshot `0` in both logs; 2 for ordering; 2 for 12; 2 for explaining batching. Common wrong answers: `render 1` (assuming `setC(c+1)` twice yields 2 *and* updater adds… the correct count is 2 total, not 3); `render 3` (treating the first two as cumulative).

R3 answer (10 points)

Initial: `['A','B','C']`; the user typed `x` into the first input (row A). New order: `['C','B','A']`. Re-render logs (no memo): `Row render C`, `Row render B`, `Row render A` **in all three strategies** (rows render in array order).

	(a) `key={i}`	**(b)** `key={l}`	**(c)** `key={Math.random()}`
Fiber identity	slot 0 stays "fiber 0" (was A, now gets `label=C`)	fibers follow labels	all old fibers discarded, new ones created
First input	still contains `x`, now next to label **C**	is empty; `x` moved with A to the **third** row	empty (state lost)
DOM nodes	no `li` moved; spans for rows 0 and 2 get text updates (`A→C`, `C→A`); row 1 untouched	**2 moved**: with new order C,B,A and old indices C=2, B=1, A=0: C stays (lastPlacedIndex becomes 2), B and A are moved (re-inserted)	3 `li` created, 3 removed
Focus	stays on first input DOM node, which is now "C's" row	the focused input follows A; moving a focused node with `insertBefore` can blur it in some browsers (browser-dependent; not observable in jsdom, where the focused input kept focus after the move; React moves B and A here, executed: removed LIB, LIA; added LI, LI)	focus lost (node destroyed)

Scoring: 3 for (a) (state stays at position; 0 moves; text updates), 4 for (b) (state follows item; 2 moves; explanation of lastPlacedIndex heuristic is conceptual — accept "1 or 2 moves" only if reasoning shown; the key is that nodes are moved, not recreated), 3 for (c) (full remount every render, focus loss). Note: whether React moves {A,B} or others is an implementation detail of the placement heuristic (conceptual: nodes keep their relative-order if already in increasing old-index order); accept the answer "nodes are moved, none created/removed" for full marks on that sub-point.

R4 answer (10 points)

(a) Mount: `Search q= d=` then `List q=`.

(b) After typing `a` (uninterrupted):

```
Search q=a d=         <- urgent render: deferred value still old; List props unchanged -> List
bails out (memo)
Search q=a d=a        <- background (transition-priority) render with new deferred value
List q=a
```

Commits: 2 (urgent commit with the input echoed; then deferred commit with the list).

(c) Typing `b` while the deferred render for `a` is in progress:

```
Search q=a d=
Search q=a d=a        <- deferred render for 'a' starts (never committed)
(List q=a)            <- only if List's function ran before the yield (timing-dependent)
Search q=ab d=        <- urgent render for the new keystroke; deferred value is still the last
COMMITTED one ('')
Search q=ab d=ab      <- restarted deferred render with the newest value
List q=ab
```

The renders with `d=a` (and possibly `List q=a`) appear in the log but are **never committed**. Commits in total: urgent(a), urgent(ab), deferred(ab) = 3. Accept `d=a` in the urgent render of `ab` only with explicit reasoning (award 1 of 3).

(d) Public guarantee: the deferred value lags and an urgent render happens first with the old value; the deferred render can be interrupted by new updates and won't commit stale intermediate values (docs). Teaching model/implementation: whether `List q=a` logged before the interrupt depends on slice timing; the lane mechanics are conceptual.

Scoring: (a) 2; (b) 3; (c) 3; (d) 2. Accept "d=a" in the urgent render of (c) only with explicit reasoning and award 1 of 3.

R5 answer (10 points)

(a) Urgent click. Render attempts `Panel id=2` → suspends; React renders the fallback and commits **one commit** (fallback inserted; existing `section` hidden via `display: none !important`). The user sees "loading". DOM: the old `section` (with the `input` and its value `hi`) remains in the document, hidden; fallback `<p>loading</p>` added. After 1000 ms: a retry render and a second commit: `p` text → `data 2`, section un-hidden, fallback removed. The input DOM node is the same, `hi` **survives** (same fiber, same position, no key change). Commits: 2. `isPending` is never true (no transition).

(b) Transition. Urgent render: `isPending=true` → commit #1 (the "updating" status appears; section unchanged). Transition render suspends; since the boundary is already revealed, no fallback is committed and the WIP is not committed. User sees the old data (`data 1`) with "updating". DOM: unchanged (no hidden style, no fallback). After resolve: commit #2: `p` text `data 2` and "updating" removed (`isPending=false` in the same commit). Total 2 commits; `hi` survives.

(c) Initial mount that suspends. Nothing existed: (a) and (b) are identical for an initial mount (no old content to hide; transitions don't apply to the initial root render): fallback shown (1 commit), then content (1 commit); no state to preserve; the user cannot have typed `hi` yet. If `id` changed later in a transition while the boundary was still showing its *fallback* (never revealed), the transition does not keep old content because none was revealed.

Scoring: (a) 3; (b) 3; (c) 2; commit counts/DOM details 2. Version note (not required): React 19 runs an extra pre-warm render after the fallback commit that is not committed (reproduced on 19.3.1). Executed on 19.3.1: (a) after the click the `section` carried `display: none !important` and `<p>loading</p>` was present, and after the load the section was un-hidden with `data 2`; (b) the old `data 1` section stayed visible with `updating`, then switched to `data 2`; in both runs it was the same `input` node and `hi` survived.

R6 answer (10 points; 2.5 each)

1. **E.** Text content differs (random per environment). Recoverable error; nearest boundary/root client-renders; final DOM text is the client's random number.
2. **W.** Attribute mismatch: dev warning with diff; no client-render; DOM keeps the **server id**. React's fiber thinks the client id. A later re-render that doesn't change the prop will not fix it. (Documented: attribute differences are not guaranteed to be patched.)
3. **M.** `useId` is deterministic and identical on both sides for the same tree position; no random value.
4. **X.** Hydration render uses `false` → `'server'` (matches server HTML); after commit the passive effect sets `true` → one extra render → DOM text becomes `client`. No error.

Scoring: 1 for classification, 1.5 for justification each.

---

8. Part 3 — Debugging answers

D1 answer (20)

Defects (each 4 points; mechanism + fix, total 20 incl. verification):

1. **Input state in a transition.** `setText` inside `startTransition` makes the input echo non-urgent. The browser already placed the character in the DOM `value`, but React's controlled value lags and, on any urgent re-render, rewrites the DOM value back to the old state: characters appear late or vanish. Rule: controlled input state must be urgent (documented caveat).
2. **Only one state**, so the filter is computed in the *urgent* render (`items.filter` on every keystroke) and `HeavyList` receives a new `visible` array every render. Need two states (`text`, `query`) or `useDeferredValue`.
3. `HeavyList` **not memoized**: it re-renders in the urgent render because the parent re-rendered. A transition without a skip does nothing for the input.
4. **Filtering runs on every render** even for unrelated updates; use `useMemo` keyed by `[items, query]` (inside the memoized consumer).
5. (Optional) **Pending UI**: spinner only; dim stale content and set `aria-busy`.

Corrected:

```tsx
function SearchPage({ items }: { items: readonly string[] }) {
  const [text, setText] = useState('');
  const [query, setQuery] = useState('');
  const [isPending, startTransition] = useTransition();
  return (
    <section aria-busy={isPending}>
      <input
        value={text}
        onChange={(e) => {
          const v = e.target.value;
          setText(v);                          // urgent
          startTransition(() => setQuery(v));  // deferred
        }}
      />
      <div style={{ opacity: isPending ? 0.6 : 1 }}>
        <HeavyList items={items} query={query} />
      </div>
    </section>
  );
}

const HeavyList = memo(function HeavyList({ items, query }: { items: readonly string[]; query: string }) {
  const visible = useMemo(() => items.filter((s) => s.includes(query)), [items, query]);
  return <ul>{visible.map((s) => <HeavyRow key={s} text={s} />)}</ul>;
});
```

Verification: Profiler (record why rendered): in the first commit after a keystroke `HeavyList` is not rendered; Chrome Performance with 4× CPU: no 400 ms task attached to the keydown; characters never revert under fast typing. Consider virtualization if the commit remains large (commit is not interruptible).

D2 answer (20)

Scoring: mechanism 6, proper fix 6, `useMemo` explanation 4, navigation issue 4.

- **Mechanism:** `fetch(...).then(...)` creates a new promise on every render; `use` suspends on it; when it resolves, React retries by calling the component again, which creates a new pending promise → endless loop (React can log "A component was suspended by an uncached promise" in dev; note that if the promise resolves within the same task the render may recover and succeed by luck, executed with an instantly resolving mock `fetch`: 3 requests then success, while with a 50 ms latency it looped (36 requests in 1.5 s)).
- `useMemo` **fails** because a component that suspends on *initial mount* never commits; its hooks state/memo cache is discarded with the abandoned render, so the memo is recreated on each attempt (and React doesn't promise `useMemo` caching semantics as a correctness guarantee).
- **Fix:** create the promise outside render with stable identity: a keyed cache (`get(id)` returns the same promise per id), or a loader in the router/parent that passes the promise as a prop, or a

Suspense-aware library:

```
const userCache = createKeyedCache<string, { name: string }>(
  (id) => fetch(`/api/users/${id}`).then((r) => {
    if (!r.ok) throw new Error(String(r.status));
    return r.json() as Promise<{ name: string }>;
  }),
);

function UserCard({ id }: { id: string }) {
  const user = use(userCache.get(id));
  return <h2>{user.name}</h2>;
}
```

(`createKeyedCache` is I3.)

- **Navigation issue:** when `id` changes via a normal update, the revealed boundary re-suspends and the whole card is hidden and replaced by "Loading…" (flicker). Fix: change `id` inside `startTransition` (route navigation) so old content stays with a pending style, and/or key the boundary if you *want* a skeleton reset. Also add an error boundary: a rejection currently has no handler.

D3 answer (20)

Defects (4 points each; 4 for corrected code, summary of verification counts within):

1. `typeof window` **branch /** `localStorage` → text/attribute difference between server (`light`) and client (stored theme). Attribute difference is *not patched*: the DOM keeps the server's `light` class (the "some users keep the light theme" symptom). Fix: determine the theme on the server from a cookie and pass it as a prop to both; or set it via a pre-hydration script on `<html>` with `suppressHydrationWarning` on that element only.
2. `new Date().toLocaleString()` → different value/locale/timezone on server and client → text mismatch → recoverable error and **client-render fallback** (the flash). Fix: render a server-provided timestamp with explicit `locale` and `timeZone`, or update after mount, or `suppressHydrationWarning` on the single `<small>` if cosmetic.
3. `<div>` **inside** `<p>`: the browser parser auto-closes `<p>`; DOM differs from React's tree → structural mismatch (and dev nesting warning). Fix: use `<div>` wrapper or `<p>` with inline content.
4. **No error reporting**: add `onRecoverableError` with release tag to detect in production; mismatches only appear for some users (theme/locale) so RUM is necessary.
5. Several console messages are React 18's multi-message output; React 19 consolidates into one message with a diff (React 19 post).

Corrected:

```
interface Props { user: { name: string } | null; theme: 'light' | 'dark'; renderedAtIso:
string }
function Page({ user, theme, renderedAtIso }: Props) {
  const stamp = new Intl.DateTimeFormat('en-GB', { dateStyle: 'medium', timeStyle: 'short',
timeZone: 'UTC' })
    .format(new Date(renderedAtIso));
  return (
    <div className={`page ${theme}`}>
      <div>Welcome {user?.name ?? 'guest'}</div>
      <small>Rendered at {stamp} UTC</small>
    </div>
  );
}
```

Verification: compare view-source vs Elements, check `CLIENT render` logs (once), no `onRecoverableError` calls in RUM, jsdom hydration test.

D4 answer (20)

Defects (scoring: 5 for in-place mutation, 5 for symptom explanations, 5 for counter and index-key issues, 5 for fix):

1. `scores.sort` **mutates the prop (which is the parent's state array)** during render. Render must not mutate props or state. The array held in `Game`'s state is silently reordered to `[30, 20, 10]` without any `setState`. `Summary`, rendered *after* `Leaderboard` in the same pass, reads the already-sorted array, so `scores[0]` is `30` (the maximum), not `10` (the first entered). Swap the sibling order and `Summary` reads the array before it is sorted and shows `10`: output depends on render order, a hallmark of impurity. After "Add 25", `[...scores, 25]` copies the *already mutated* array, so the original insertion order is lost permanently.
2. **Module-level** `renderCount++` is a render side effect; StrictMode double invocation in development increments it twice per render (the "jumps by 2" symptom), and abandoned or repeated concurrent renders would also increment it. Use the Profiler or a ref updated in an effect for diagnostics; never display it from render-time mutation.
3. **Index keys** on a list that is re-sorted: state in `li` (none here) would stay attached to positions; use a stable key (the score isn't unique; carry ids).
4. **"First entered" requires keeping entry order** distinct from display order: store `{ id, value }` entries in insertion order and derive the sorted list.

Corrected:

```tsx
interface Entry { id: number; value: number }

function Leaderboard({ entries }: { entries: readonly Entry[] }) {
  const sorted = useMemo(() => [...entries].sort((a, b) => b.value - a.value), [entries]);
  return <ol>{sorted.map((e) => <li key={e.id}>{e.value}</li>)}</ol>;
}
function Summary({ entries }: { entries: readonly Entry[] }) {
  return <p>First entered: {entries[0]?.value}</p>;
}
```

Verification: run with StrictMode on; results identical in both passes; freeze the props in dev (`Object.freeze`) to catch mutation (it throws in strict-mode code); a test that renders twice and compares output.

D5 answer (20)

Scoring: explanation 5, three fixes with trade-offs 10, no-fix case 5.

- **Why memo fails:** `options` is a new object literal each render and `onSelect` a new function each render → shallow-equality (`Object.is` per prop) sees changes → the memo bailout never applies. The parent re-renders on every mousemove because `hover` state lives in `Dashboard`.
- **Fix A (structural, no memoization):** move `hover` state down into a `HoverLayer` that wraps only `Tooltip` and the mouse listener target, or use CSS `:hover` / refs for tooltips; `Chart` isn't a descendant of the changing state, so it doesn't re-render. Trade-off: restructure layout/event delegation.
- **Fix B (stable references):** hoist `options` to a module constant (or `useMemo` keyed on actual inputs), `onSelect` via `useCallback` with deps `[rows]` (or a ref-based stable handler); keep `memo` on `Chart`. Trade-off: dependency maintenance; stale closures if deps wrong; `rows` changing invalidates anyway.
- **Fix C (React Compiler):** with the compiler enabled and code following the Rules of React, the compiler memoizes values/callbacks automatically so `Chart`'s props may be referentially stable; verify with the compiler's DevTools badge ("Memo ✨") and the current compiler docs; trade-off: build integration, bailouts for non-conforming code, still requires purity.
- **Do not fix when:** `Chart` render is cheap (≈ ms), the mousemove rate is low, or the component renders canvas through an effect that is idempotent and cheap; profiling shows no frame drops. Memoization adds comparison cost and complexity; measure first.

---

9. Part 4 — Implementation reference solutions

I1 reference (25)

Scoring: recorder 6, spy test 8, Profiler test 6, explanation 5.

```ts
// recorder.ts
import type { ProfilerOnRenderCallback } from 'react';

export interface CommitRecord {
  id: string;
  phase: 'mount' | 'update' | 'nested-update';
  actualDuration: number;
}

export function createRecorder() {
  const commits: CommitRecord[] = [];
  const onRender: ProfilerOnRenderCallback = (id, phase, actualDuration) => {
    commits.push({ id, phase, actualDuration });
  };
  return { onRender, commits, reset: () => { commits.length = 0; } };
}
```

```tsx
// memo.test.tsx
import { Profiler, memo, useState } from 'react';
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { expect, test, vi } from 'vitest';
import '@testing-library/jest-dom/vitest';   // needed for toHaveTextContent (or assert on .textContent)
import { createRecorder } from './recorder';

test('memo child is skipped when parent state changes', async () => {
  const childBody = vi.fn();
  const rec = createRecorder();

  const Child = memo(function Child({ label }: { label: string }) {
    childBody(label);
    return <span>{label}</span>;
  });

  // The Profiler itself must sit behind a memo boundary. If <Profiler> is rendered directly by the
  // re-rendering Parent, React re-renders the Profiler fiber and calls onRender on every update
  // (with actualDuration ~ 0) even though Child bails out.
  const ProfiledChild = memo(function ProfiledChild() {
    return (
      <Profiler id="child" onRender={rec.onRender}>
        <Child label="static" />
      </Profiler>
    );
  });

  function Parent() {
    const [n, setN] = useState(0);
    return (
      <>
        <button onClick={() => setN(n + 1)}>{n}</button>
        <ProfiledChild />
      </>
    );
  }

  render(<Parent />);
  expect(childBody).toHaveBeenCalledTimes(1);
  expect(rec.commits.map((c) => c.phase)).toEqual(['mount']);
  rec.reset();
```

```
  await userEvent.click(screen.getByRole('button'));
  expect(screen.getByRole('button')).toHaveTextContent('1');
  expect(childBody).toHaveBeenCalledTimes(1);   // (a) spy: body not invoked
  expect(rec.commits).toHaveLength(0);          // (b) Profiler: no commit inside the profiled
subtree
});
```

What each proves: (a) the child's function body did not run for that update (render-phase evidence); (b) no commit touched the profiled subtree (`onRender` is documented to fire when components within the profiled tree update). Executed on react 19.3.1 + Testing Library 16.3.3 + Vitest 3 with `@testing-library/jest-dom/vitest` imported: this version passes; the earlier form with `<Profiler>` placed directly inside `Parent` **fails** assertion (b) (`onRender` fired once with `actualDuration` about 0.01 ms because React re-renders the Profiler fiber itself, `updateProfiler` sets its `Update` flag unconditionally), which is itself a worthwhile finding to report. What they cannot prove: that the *parent* did not do expensive work; that behavior holds in StrictMode (the spy would count 2 on mount); production timing; that `memo` is *worth* its cost. Evaluation notes: uses `vi.fn` not a module counter; resets between phases; asserts mount phase first; recognizes Profiler is disabled in production builds unless profiling build.

I2 reference (25)

Scoring: urgent echo 5, transition + memo 8, a11y 5, test 4, browser proof 3.

```tsx
import { memo, useMemo, useState, useTransition, type ChangeEvent } from 'react';

const Results = memo(function Results({ items, query }: { items: readonly string[]; query:
string }) {
  const q = query.trim().toLowerCase();
  const visible = useMemo(() => items.filter((s) => s.toLowerCase().includes(q)), [items, q]);
  return <ul aria-label="Results">{visible.map((s) => <li key={s}>{s}</li>)}</ul>;
});

export function FilterableList({ items }: { items: readonly string[] }) {
  const [text, setText] = useState('');
  const [query, setQuery] = useState('');
  const [isPending, startTransition] = useTransition();

  const onChange = (e: ChangeEvent<HTMLInputElement>) => {
    const v = e.target.value;
    setText(v);
    startTransition(() => setQuery(v));
  };

  return (
    <div aria-busy={isPending}>
      <label>
        Filter
        <input value={text} onChange={onChange} />
      </label>
      <p role="status" aria-live="polite">{isPending ? 'Updating results' : ''}</p>
      <div style={{ opacity: isPending ? 0.6 : 1, transition: 'opacity 150ms' }}>
        <Results items={items} query={query} />
      </div>
    </div>
  );
}
```

```tsx
// FilterableList.test.tsx
import { render, screen, waitFor } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { expect, test } from 'vitest';
import { FilterableList } from './FilterableList';

test('filters results after typing', async () => {
  render(<FilterableList items={['apple', 'apricot', 'banana']} />);
  await userEvent.type(screen.getByRole('textbox'), 'ap');
  await waitFor(() => {
    const items = screen.getAllByRole('listitem').map((li) => li.textContent);
    expect(items).toEqual(['apple', 'apricot']);
  });
});
```

Browser proof: Profiler with "Record why each component rendered": after a keystroke the first commit shows `FilterableList` rendered and `Results` not rendered; the second commit renders `Results`. Chrome Performance (4× CPU): keydown task contains only the urgent render; list work appears as multiple short tasks. The live region is rendered permanently (so it can announce) with empty content when idle. Criteria: `text` and `query` separated; `memo` on consumer; no `startTransition` around `setText`; `role="status"` stays mounted; test uses `waitFor`.

I3 reference (25)

Scoring: cache semantics 9, error boundary + Suspense composition 8, retry flow 5, explanation 3.

```ts
// keyed-cache.ts
export interface KeyedCache<K, V> {
  get(key: K): Promise<V>;
  preload(key: K): void;
  invalidate(key?: K): void;
}

export function createKeyedCache<K extends string | number, V>(
  loader: (key: K) => Promise<V>,
): KeyedCache<K, V> {
  const map = new Map<K, Promise<V>>();
  return {
    get(key) {
      let p = map.get(key);
      if (!p) {
        p = loader(key);
        p.catch(() => {});          // mark handled; consumers still see the rejection via use()
        map.set(key, p);
      }
      return p;
    },
    preload(key) { void this.get(key); },
    invalidate(key) { if (key === undefined) map.clear(); else map.delete(key); },
  };
}
```

```tsx
// Section.tsx
import { Component, Suspense, use, type ReactNode } from 'react';
import { createKeyedCache } from './keyed-cache';

const users = createKeyedCache<string, { name: string }>((id) =>
  fetch(`/api/users/${id}`).then((r) => {
    if (!r.ok) throw new Error(`HTTP ${r.status}`);
    return r.json() as Promise<{ name: string }>;
  }),
);

interface BoundaryProps { onReset: () => void; children: ReactNode }
interface BoundaryState { error: Error | null }

class Boundary extends Component<BoundaryProps, BoundaryState> {
  state: BoundaryState = { error: null };
  static getDerivedStateFromError(error: Error): BoundaryState { return { error }; }
  retry = () => { this.props.onReset(); this.setState({ error: null }); };
  render() {
    return this.state.error ? (
      <div role="alert">
        Failed: {this.state.error.message} <button onClick={this.retry}>Retry</button>
      </div>
    ) : this.props.children;
  }
}

function User({ id }: { id: string }) {
  const user = use(users.get(id));
  return <h2>{user.name}</h2>;
}

export function Section({ id }: { id: string }) {
  return (
    <Boundary onReset={() => users.invalidate(id)}>
      <Suspense fallback={<p>Loading…</p>}>
        <User id={id} />
      </Suspense>
    </Boundary>
  );
}
```

Explanation: if a rejected promise were evicted automatically at rejection, React's retry (including its one immediate recovery re-render after an error) would call `get(key)` again, create a *new* pending promise, suspend, and the boundary could oscillate between loading and error, hammering the endpoint. Keeping the rejected promise until an explicit user-driven `invalidate` makes the error state stable and the retry

deliberate. Also acceptable: backoff-based retry in the data layer. Unbounded growth: mention eviction/TTL for production caches. Criteria: stable identity, `catch` to avoid unhandled-rejection noise, error boundary outside Suspense, retry resets both cache and boundary.

I4 reference (25)

Scoring: hook 8, stable subscribe 4, test 9, explanation 4.

```
// use-media-query.ts
import { useCallback, useSyncExternalStore } from 'react';

export function useMediaQuery(query: string, serverValue = false): boolean {
  const subscribe = useCallback(
    (onChange: () => void) => {
      const mql = window.matchMedia(query);
      mql.addEventListener('change', onChange);
      return () => mql.removeEventListener('change', onChange);
    },
    [query],
  );
  return useSyncExternalStore(
    subscribe,
    () => window.matchMedia(query).matches,
    () => serverValue,
  );
}
```

```tsx
// use-media-query.test.tsx
// @vitest-environment jsdom
import { act } from 'react';
import { hydrateRoot } from 'react-dom/client';
import { renderToString } from 'react-dom/server';
import { afterEach, beforeEach, expect, test, vi } from 'vitest';
import { useMediaQuery } from './use-media-query';

(globalThis as { IS_REACT_ACT_ENVIRONMENT?: boolean }).IS_REACT_ACT_ENVIRONMENT = true;

function Layout() {
  const wide = useMediaQuery('(min-width: 600px)', false);
  return <p>{wide ? 'wide' : 'narrow'}</p>;
}

beforeEach(() => {
  window.matchMedia = ((q: string) => ({
    matches: true, media: q,
    addEventListener: () => {}, removeEventListener: () => {},
  })) as unknown as typeof window.matchMedia;
});
afterEach(() => { document.body.innerHTML = ''; vi.restoreAllMocks(); });

test('hydrates with server value then converges without errors', async () => {
  const html = renderToString(<Layout />);
  expect(html).toContain('narrow');                      // server value

  const container = document.createElement('div');
  container.innerHTML = html;
  document.body.append(container);

  const errors: unknown[] = [];
  vi.spyOn(console, 'error').mockImplementation(() => {});
  await act(async () => {
    hydrateRoot(container, <Layout />, { onRecoverableError: (e) => errors.push(e) });
  });

  expect(errors).toHaveLength(0);
  expect(container.textContent).toBe('wide');            // client snapshot applied after
hydration
});
```

Explanation: `subscribe` must be stable because `useSyncExternalStore` resubscribes whenever the `subscribe` identity changes; an inline function would unsubscribe/subscribe on every render (and could miss events between). Also: `getSnapshot` must return an immutable, cached value for an unchanged store (a boolean is fine); returning a fresh object each call triggers an infinite-loop warning. During

hydration React uses `getServerSnapshot`, and after hydration it re-checks `getSnapshot()` and re-renders if it differs. `renderToString` in a jsdom environment is acceptable for a lab; in real SSR, `window` does not exist, which is why `getSnapshot` is not called there.

---

10. Part 5 — Architecture answers

R7 evaluation (14 points)

Rubric: state placement 3; concurrency features chosen with justification 3; boundaries 3; ticker 2; SSR/measurement 2; two "don't use" decisions 1.

Example well-reasoned answer:

- **State placement.** Filter state is URL-driven (search params) with a local input echo state (`text`) per control; the applied filter is a separate state updated in a transition. Widget data is keyed by the applied filter. This keeps the filter bar urgent and widgets deferrable.
- **Transitions/deferral.** Filter application and route navigation run in `startTransition`; widgets receive the applied filter and are `memo`'d or placed so they do not re-render on text echo. `useDeferredValue` is used inside the table widgets only if their parent cannot be wrapped (e.g. value arrives from a context).
- **Suspense.** One boundary per widget (fixed-size skeletons to avoid layout shift) and one at route level for the page shell; transitions keep revealed widgets visible while new data loads. Error boundaries per widget (blast radius = widget) with a retry that invalidates that widget's cache entry.
- **Memoization.** Canvas chart components `memo`'d with stable `options`; tables virtualized (5,000 rows × columns would otherwise create a huge commit); KPIs trivial.
- **Ticker.** `useSyncExternalStore` with a throttled store (batch updates per animation frame) feeding only ticker tiles; those updates are synchronous by design and not transitions; isolate them in their own subtree so they do not re-render widgets.
- **SSR/hydration.** Server-render the layout skeleton and filter bar; stream widgets behind Suspense; avoid time/locale dependence (explicit `Intl` options), `useId` for ids; selective hydration prioritizes the filter bar the user touches first.
- **Measurement.** Profiler traces (commit counts, "why rendered"), Chrome Performance with CPU throttling for INP on filter interaction, RUM INP/CLS, error-rate per widget, and `onRecoverableError` count.
- **Not using concurrent features:** (1) the ticker (must be consistent and synchronous; a transition would cause tearing/lag); (2) the date-picker open/close and text echo (urgent by nature). Also acceptable: small KPI tiles where a transition adds extra renders for no gain.

R8 evaluation (13 points)

Rubric: per-element decision table 6; mismatch risks and mitigations 3; selective hydration/streaming 2; reporting/skew 2.

Element	Decision	Risk	Mitigation
Price by country (cookie)	Server-rendered from the cookie/header	Client first render must use the same country source; `Intl` default locale differs	Pass `country` /currency/locale as props from the server; explicit `Intl` options; vary cache by country
"N people viewing"	After mount (or streamed in a Suspense boundary from a realtime source)	Random/live number differs	Placeholder-sized skeleton; update in effect/ `useSyncExternalStore` with `getServerSnapshot`
Cart count in `localStorage`	After mount	`localStorage` unavailable on server; value differs	`useSyncExternalStore` with `getServerSnapshot` (0 or hidden); or move to cookie/server session
A/B variant (cookie)	Server-rendered from cookie; same value passed to client	Cached HTML served to wrong variant	`Vary: Cookie` / per-variant cache keys / edge assignment; never re-assign client-side during hydration
Reviews (slow)	Streamed behind Suspense with fixed-height skeleton	Boundary mismatch isolates damage	Separate boundary; error boundary; prefetch on server
Sale countdown	After mount, based on a server-provided end timestamp	`Date.now()` differs	Render the end timestamp text statically; tick after mount; `suppressHydrationWarning` only on the ticking text if needed

Selective hydration: the buy button region must hydrate early (not behind the slow reviews); a click on an un-hydrated boundary makes React hydrate that boundary synchronously and then dispatch the click (not a queued replay; if the boundary's code is not loaded the click is dropped); keeping reviews in a Suspense boundary prevents blocking hydration of the rest. Reporting: `onRecoverableError` (+ `onCaughtError/onUncaughtError`, React 19 `hydrateRoot` options) tagged with release id; alert on spike after deploy; skew mitigations: atomic deployment of HTML and assets, HTML cache TTL shorter than asset retention, hashed assets retained for old HTML, sample component stacks.

R9 evaluation (13 points)

Rubric: investigation procedure 4; decision criteria 4; compiler note 2; regression prevention 1; counter-example 2.

Example answer:

- **Procedure.** Define the symptom first (INP, dropped frames, CPU) with RUM/lab numbers; reproduce with Performance panel (CPU throttled) and React Profiler (commits, "why did this render", render durations); find the highest *cost* (duration × frequency), not the highest count; check if commit/DOM or render function dominates; classify cause: state too high, unstable props, context churn, expensive computation, too many nodes.
- **Decision criteria.** Prefer (1) move state down / lift content up (structure); (2) split context; (3) make computations cheaper or virtualize; (4) `memo` / `useMemo` / `useCallback` where the profile shows an

expensive subtree re-rendering with props that *can* be stable; (5) accept re-renders that cost < ~1 ms and don't affect interaction. Render count alone is not a defect.

- **Compiler.** With the React Compiler (1.0.0 on npm as of 2026-10-22; check the current docs and your build constraints), automatic memoization reduces the need for manual memo, but it assumes the Rules of React; code that mutates or reads refs during render can be skipped; manual memo remains for semantic needs (effect dependency stability, expensive third-party). Policy: adopt incrementally, keep profiling.
- **Regression prevention.** Performance budgets and interaction tests in CI (e.g. INP in lab, bundle size), lint for impure render and hook deps, Profiler-based tests for hot paths, code review guidance, dashboards.
- **Memo worse:** memoizing cheap leaf components adds comparison cost per render, retains closures/memory, and obscures code; `useMemo` around trivial calculations; memoized component receiving a fresh `children` element prop or inline object defeats memo while paying comparisons; wrapping everything also makes stale closure bugs likelier.

---

11. Scoring rubric and promotion

11.1 Score sheet

Category	Items	Max	Your score	%
Explanation	E1–E10	100		
Reasoning	R1–R6 (60) + R7–R9 (40)	100		
Debugging	D1–D5	100		
Implementation	I1–I4	100		
Total		**400**		

11.2 Pass criteria (from ROADMAP)

- **≥ 80% overall (≥ 320/400)** *and*
- **≥ 70% in each category (≥ 70/100)** — explanation, implementation, debugging, reasoning.

A learner who passes the conceptual part (Explanation) but not the others does **not** advance.

11.3 Critical-misconception rule

If your written answers assert any of the following as true, subtract 10 points from the relevant category (maximum one deduction per statement) and re-study the linked lesson even if you pass:

Statement	Lesson
"Rendering means the DOM was updated."	02, 11
"`startTransition` runs code on another thread or delays the callback."	13, 35
"`useEffect` runs on the server."	18, 26

Statement	Lesson
"React diffs the real DOM."	04, 05
"Hydration patches attribute mismatches."	25, 37
"A suspended component keeps its state on initial mount."	24, 36

11.4 Remediation map

Weak area (below 70% in category, or missed items)	Re-study
E1, R1	02, 11, 18, 19, 20, 21
E2, E3, R3	04, 05, 06, 07, 34
E4, E5, D4	08, 09, 10, 12, 28
E6, R4, D1, I2	13, 14, 15, 16, 23, 35
E7, R1	18, 19, 20, 21
E8, R2	17, 22
E9, R5, D2, I3	24, 36
E10, R6, D3, I4, R8	25, 26, 27, 37
D5, R9, I1	29, 30, 32, 33
Source-reading claims (E6 point 5)	31

11.5 Promotion criteria

You may begin **Phase 3 — State and Hooks** when **all** hold:

1. You meet the pass criteria in §11.2 on a first attempt, or on a retake after at least 7 days of remediation (retakes use new data/seeds in the predictions — write your own variants to test the same concepts).
2. You can reproduce, without notes, the timelines of Scenarios D, E and F (lessons 35–37) on a whiteboard, naming for each step what is rendered, committed, discarded and what the DOM contains.
3. Your instrumented examples (I1) run and are kept as a personal toolkit for Phase 3: a render-count recorder, a Profiler workflow, and a hydration test harness.
4. You have recorded at least one real Profiler session of an app of your own and explained one unnecessary re-render in terms of triggers (lesson 01), reconciliation (06/07) and bailouts (29).

If you do not pass: follow the remediation map, redo only the failed categories after 7 days, and reattempt a fresh set of prediction questions you write yourself for the weak lessons.

⚠ **Version note.** Items marked ⚠ (Suspense sibling pre-warming, hydration error message formatting, React Compiler, Profiler/performance tracks, `useId` formats; the `useId` prefix changed `:r:` -> `«r»` -> `_r_` across 19.0/19.1/19.2) are behaviors that may change between React 19.x minor versions. Where your observation differs from the key on a ⚠ item, check the current React release notes and docs before concluding you were wrong; the answer key marks which statements are guarantees and which are implementation details.